Forward vs Reverse Proxy

egress control, TLS termination and NGINX

Watch the 3-min video

In one line: a forward proxy acts for clients, controlling and hiding their outbound traffic; a reverse proxy acts for servers, sitting in front of them, hiding the backends and taking work off the app.

1 Two machines in the middle

Two kinds of machines sit in the middle of web traffic, and both are called **proxies**. One works for the clients. The other works for the servers. Mixing them up is a classic interview slip.

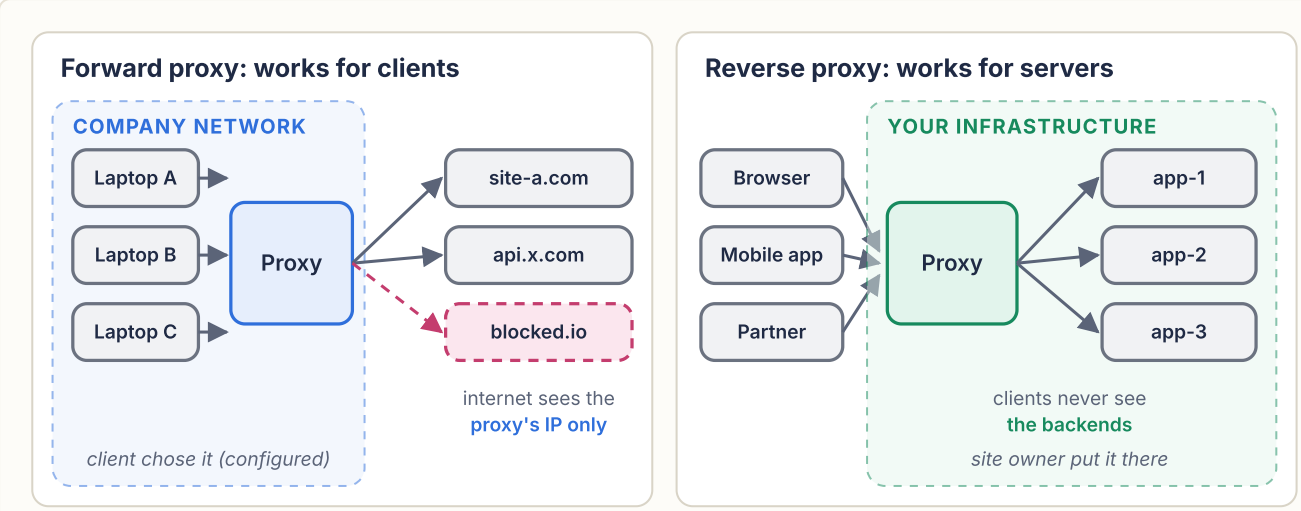


Figure 1. Left: laptops send outbound requests through a forward proxy, which can block sites. Right: every client hits the reverse proxy, which forwards to a backend the client never sees.

2 Forward proxy: egress control for clients

A **forward proxy** sits in front of clients. Laptops inside a company send their outbound requests to the proxy, and the proxy talks to the internet on their behalf. Websites see the proxy's address, not the employee's.

Companies use it for **egress control**: blocking certain sites, enforcing **allow-lists** so servers can only call approved APIs (a payment provider, a package mirror), **logging** who called what, and **caching** shared downloads. Squid is a classic example. In the cloud, an egress proxy or a firewall in front of a **NAT gateway** plays the same role: the NAT gives private servers one public address, and the firewall decides which domains they may reach.

3 Reverse proxy: the front door for servers

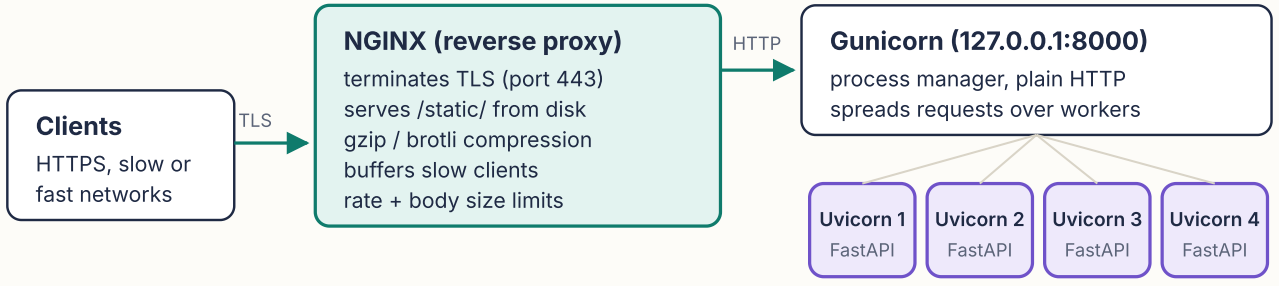
A **reverse proxy** sits in front of servers. Clients think they are talking to your site, but they are really talking to the proxy, which forwards each request to one of your backends. The client never sees the backends.

Ask	Forward proxy	Reverse proxy
Who put it there?	The client side (browser or OS setting, corporate policy)	The site owner; clients don't know it exists
Whose identity does it hide?	Hides the clients from servers	Hides the servers from clients
Typical jobs	Block sites, allow-list APIs, log, cache	TLS, routing, load balancing, caching, compression
Examples	Squid, corporate web proxy, cloud egress firewall	NGINX, HAProxy, Envoy, API gateway, cloud load balancer

A reverse proxy takes a lot of work off app servers:

- **TLS termination**: the proxy holds the certificates and decrypts HTTPS, so apps don't.
- **Routing** by host or path (`api.` to one service, `/images/` to another) and **load balancing** across instances.
- **Caching** static files and responses, and **compression** with gzip or brotli.
- **Protection**: rate limiting, request size limits, and **buffering slow clients**.

4 The classic setup: NGINX, Gunicorn, Uvicorn



Python workers only see fast, complete, plain-HTTP requests for dynamic pages

Figure 2. NGINX faces the internet and does the generic work. It forwards only dynamic requests to Gunicorn on a local port, which spreads them across Uvicorn worker processes running the FastAPI app.

WORKED EXAMPLE: WHY BUFFERING SLOW CLIENTS MATTERS

Your app runs 4 Uvicorn workers. A user on a weak mobile link uploads a 2 MB form at 100 KB/s: that is **20 seconds** of sending. Without a proxy, a worker holds that connection for the whole upload. NGINX reads the full request body first (`client_max_body_size` rejects anything too large), then hands Gunicorn the complete request over localhost in about a millisecond. On the way back, NGINX buffers the response and trickles it to the slow client, so the Python worker is free again almost at once.

Config sketch: `location /static/ { root /srv/app; }` and `location / { proxy_pass http://127.0.0.1:8000; proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for; }`

5 Common reverse proxies

NGINX, **HAProxy** and **Envoy** are the usual choices. Envoy adds features for microservices: retries, circuit breaking, rich metrics, and dynamic configuration, and it is the data plane behind service meshes such as **Istio**. **API gateways** and **cloud load balancers** (AWS ALB, Azure Application Gateway) are reverse proxies too, with extra features such as authentication and API keys on top.

COMMON MISTAKES

- Calling a load balancer a “forward proxy”: it acts for the servers, so it is a reverse proxy.
- Logging the proxy's IP as the client IP: read `X-Forwarded-For`, but only trust it from your own proxy.
- Exposing Gunicorn directly on the internet: no TLS, and slow clients tie up workers.
- Assuming a NAT gateway alone controls egress: it hides addresses but does not filter by domain.

6 Key terms

Forward proxy: acts for clients' outbound traffic

Egress control: limiting what inside systems can reach

X-Forwarded-For: header carrying the original client IP

Reverse proxy: acts for servers, in front of them

TLS termination: decrypt HTTPS at the proxy

Service mesh: sidecar proxies (Envoy) per service

7 Interview questions

Q1. What's the difference between a forward and a reverse proxy? BASIC

A forward proxy acts for clients: they send outbound traffic through it, and it controls and hides them. A reverse proxy acts for servers: it sits in front of them, receives all client traffic and hides the backends. Ask who put it there and whose identity it hides.

Q2. What can a reverse proxy offload from app servers? BASIC

TLS termination, routing by host or path, load balancing, caching, gzip or brotli compression, rate limiting, request size limits, and buffering slow clients.

Q3. Your servers must call only Stripe and your package mirror. How do you enforce it? SCENARIO

Put outbound traffic through an egress proxy or a cloud firewall with an allow-list of domains, block direct internet access in the network rules, and log every request. Deny by default; add domains by review.

Q4. Why put NGINX in front of Gunicorn and Uvicorn? SCENARIO

NGINX terminates TLS, serves static files from disk, compresses, enforces limits and buffers slow clients. With 4 workers, four slow uploads at 20 s each would otherwise block the whole app; with NGINX, each worker is busy for milliseconds.

Q5. After adding a reverse proxy, every request in your logs shows the same client IP. Why? SCENARIO

The app now sees the proxy's address. Have the proxy set `X-Forwarded-For` (or the standard `Forwarded` header) and configure the app to trust it only from the proxy's IP, so clients can't fake it.

Q6. Is a load balancer a forward or a reverse proxy? TRICKY

A reverse proxy. The site owner runs it, clients don't know the backends behind it, and it spreads requests across them. API gateways are reverse proxies too.

Q7. Can a corporate forward proxy read your HTTPS traffic? TRICKY

Normally no. The browser sends `CONNECT host:443` and the proxy only tunnels encrypted bytes, so it sees the hostname, not the content. It can read traffic only with TLS interception: the company installs its own root certificate on the laptops and the proxy re-encrypts.

Next: SD-B-018 · CDNs Explained backendroadmap.com